

Pipeline con Pandas y Matplotlib — Analizador de Ventas

Actividad Práctica · Nivel Intermedio · VS Code + GitHub Actions + Cobertura

NIVEL INTERMEDIO
— Pandas · Matplotlib · pytest-cov

Nombre del alumno/a	ANGEL ECHENIQUE
Fecha	28/03/26
Repositorio GitHub	https://github.com/kindred-98/Pipeline_Ventas.git

Instrucciones: Trabaja con datos reales en formato CSV usando Pandas y Matplotlib. El pipeline comprobará automáticamente la calidad del código, los tests Y la cobertura mínima. Cita las fuentes de las librerías si las consultas.

Objetivo: Construir un analizador de ventas con Pandas y Matplotlib, testearlo con pytest, medir la cobertura de código con pytest-cov y crear un pipeline GitHub Actions que bloquee si la cobertura baja del 80%.

Contexto: Pandas es la librería de análisis de datos más usada en Python. Matplotlib permite crear gráficos a partir de datos. pytest-cov mide qué porcentaje de tu código está cubierto por tests. Todas se instalan con pip.

FASE 1 — Entorno, estructura y datos de prueba

✓	Paso	Cómo verificarlo
✓✓	Crea repo GitHub público: pipeline-ventas	Visible en tu perfil
✓✓	Clona en VS Code, crea y activa entorno virtual	(venv) en terminal
✓✓	Instala librerías: pip install pandas matplotlib pytest pytest-cov	pip list muestra las 4
✓✓	Crea requirements.txt:	Archivo con las 4 librerías
✓✓	Crea la estructura de carpetas indicada abajo	Carpetas visibles en VS Code



1.1 — Estructura del proyecto

pipeline-ventas/		
—	src/	
	└─ analizador.py	←módulo principal
—	tests/	
	└─ test_analizador.py	←tests pytest
—	data/	
	└─ ventas.csv	←datos de prueba
—	.github/workflows/pipeline.yml	←pipeline CI
—	requirements.txt	
└─	README.md	

1.2 — Crea el fichero de datos data/ventas.csv

Copia exactamente este contenido en data/ventas.csv (las comas son el separador):

producto,cantidad,precio_unitario,mes
Manzanas,50,1.20,enero
Naranjas,30,0.90,enero
Plátanos,80,0.60,enero
Manzanas,40,1.25,febrero
Naranjas,55,0.95,febrero
Plátanos,90,0.65,febrero
Manzanas,60,1.30,marzo
Naranjas,20,1.00,marzo
Plátanos,75,0.70,marzo

**FASE 2 — Desarrolla el módulo de análisis con Pandas**

Implementa el archivo `src/analizador.py` con las siguientes funciones. Lee los docstrings antes de implementar:

```
# src/analizador.py

import pandas as pd
import matplotlib
matplotlib.use('Agg')    # modo sin pantalla (necesario en el pipeline)
import matplotlib.pyplot as plt

def cargar_ventas(ruta_csv: str) -> pd.DataFrame:
    """Carga el CSV y devuelve un DataFrame.
    Lanza FileNotFoundError si el archivo no existe.
    Lanza ValueError si le faltan columnas obligatorias."""
    pass

def calcular_total_por_producto(df: pd.DataFrame) -> pd.Series:
    """Devuelve Series con el total (cantidad × precio_unitario) por producto."""
    pass

def top_productos(df: pd.DataFrame, n: int = 3) -> list:
    """Devuelve lista con los n productos con mayor total de ventas."""
    pass

def ventas_por_mes(df: pd.DataFrame) -> pd.Series:
    """Devuelve Series con el total de ventas agrupado por mes."""
    pass

def generar_grafico_barras(df: pd.DataFrame, ruta_salida: str) -> None:
    """Genera gráfico de barras con ventas por producto y lo guarda en ruta_salida."""
    pass
```

2.1 — Antes de codificar: analiza las funciones

Pregunta	Tu respuesta
¿Qué columna se multiplica para obtener el total de ventas?	Total=cantidad×precio_unitario
¿Qué método de Pandas usarías para agrupar por producto?	Groupby() en Pandas porque necesitas aplicar el patrón clásico de análisis de datos tipo split-apply-combine: 1. Divide (split) el DataFrame en grupos según valores únicos de producto 1. Grupo A →filas donde producto = A 2. Grupo B →filas donde producto = B 2. Luego tú decides qué hacer con cada grupo (apply) 3. Y finalmente Pandas lo recompone (combine)
¿Qué excepción lanzas si el archivo CSV no existe?	FileNotFoundError Esto ocurre automáticamente si pd.read_csv() no encuentra el archivo..
¿Por qué se usa matplotlib.use('Agg') en el pipeline?	Porque en GitHub Actions no existe un entorno gráfico. Matplotlib normalmente intenta abrir una ventana para mostrar gráficos, pero en un servidor CI/CD.
¿Qué devuelve top_productos si n=2 con nuestros datos?	Manzanas = 188 Plátanos = 159

FASE 3 — Genera y completa los tests con IA (cobertura $\geq$ 80%)

Tip IA: Pégle a la IA el código completo de `analizador.py` y pídele tests con `pytest` para cada función. Indica que deben usar el CSV de prueba y alcanzar cobertura mínima del 80%. Usa fixtures de `pytest` para el `DataFrame`.

3.1 — Prompt para la IA: escríbelo aquí antes de enviarlo

Genera tests unitarios en `pytest` para el módulo `analizador.py` que te paso a continuación. El proyecto contiene un archivo CSV en `data/ventas.csv` con las columnas: `producto`, `cantidad`, `precio_unitario`, `mes`.

Necesito tests para todas las funciones del módulo: `cargar_ventas`, `calcular_total_por_producto`, `top_productos`, `ventas_por_mes` y `generar_grafico_barras`.

Requisitos:

- Usa fixtures de `pytest` para cargar el `DataFrame` desde el CSV.
- Usa formato AAA (Arrange, Act, Assert).
- Incluye tests de éxito y tests de error.
- Asegura una cobertura mínima del 80% usando `pytest-cov`.
- Para `generar_grafico_barras`, verifica que el archivo PNG se crea en disco.
- No inventes columnas ni datos que no existan en el CSV.
- No repitas tests innecesarios.

Y le eh pegado el codigo: del `analizador.py`

3.2 — Estructura mínima de tests que debes tener

Verifica que tienes al menos estos tests. Marca los que hayas implementado:

✓	Test requerido	¿Lo generó la IA o lo escribiste tú?	
✓✓	<code>test_cargar_ventas_ok</code> : el CSV se carga correctamente	IA	
✓✓	<code>test_cargar_ventas_archivo_no_existe</code> : lanza <code>FileNotFoundError</code>	IA	
✓✓	<code>test_cargar_ventas_columna_faltante</code> : lanza <code>ValueError</code>	IA	
✓✓	<code>test_calcular_total_manzanas</code> : el total de Manzanas es correcto	IA	
✓✓	<code>test_top_productos_n2</code> : devuelve exactamente 2 productos	IA	
✓✓	<code>test_ventas_por_mes_enero</code> : total de enero es correcto	IA	
✓✓	<code>test_generar_grafico_crea_archivo</code> : el PNG se genera en disco	IA	

3.3 — Ejecuta tests y mide la cobertura en local

✓	Comando	Qué debes ver
✓✓	<code>pytest tests/ -v</code>	Tests en verde
✓✓	<code>pytest tests/ -v --cov=src --cov-report=term-missing</code>	Tabla de cobertura con %
✓✓	Cobertura $\geq$ 80% en <code>analizador.py</code>	Número en verde en la columna TOTAL
✓✓	Si $<$ 80%: pide a la IA que genere tests para las líneas no cubiertas	Cobertura sube al 80%+

¿Qué líneas quedaban sin cubrir y qué tests añadiste para cubrirlas?

Las líneas sin cubrir pertenecían a la validación de columnas en `cargar_ventas` y a la rama alternativa de `top_productos` cuando `n` es mayor que el número de productos.

Añadí un test para CSV con columnas incompletas y otro para `top_productos(n=10)` para cubrir esas rutas.

**FASE 4 — Pipeline con cobertura mínima obligatoria**

Crea el archivo `.github/workflows/pipeline.yml`. Este pipeline bloqueará el despliegue si la cobertura baja del 80%:

name: Pipeline Ventas con Cobertura
on:
push:
branches: [main]
pull_request:
branches: [main]
jobs:
test-and-coverage:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: actions/setup-python@v5
with:
python-version: '3.11'
- name: ☐ Instalar dependencias
run: pip install -r requirements.txt
- name: ☐ Ejecutar tests + cobertura mínima 80%
run: pytest tests/ -v --cov=src --cov-fail-under=80
--cov-fail-under=80 →BLOQUEA si cobertura < 80%
- name: ☐ Generar reporte de cobertura HTML



```
run: pytest tests/ --cov=src --cov-report=html
```

```
- name: Subir reporte como artefacto
```

```
uses: actions/upload-artifact@v4
```

```
with:
```

```
name: cobertura-html
```

```
path: htmlcov/
```

4.1 — Sube y analiza el pipeline

✓	Paso	Qué debes ver
✓✓	git add . git commit -m 'feat: analizador ventas + tests + pipeline cobertura' git push	Pipeline se activa en la pestaña Actions
✓✓	Observa la etapa 'Ejecutar tests + cobertura mínima 80%'	Muestra % de cobertura y pasa o falla
✓✓	Descarga el artefacto 'cobertura-html' y abre htmlcov/index.html en el navegador	Ves las líneas cubiertas en verde y las no cubiertas en rojo

¿Pasó el pipeline a la primera? ¿Qué cobertura obtuvo? ¿Qué etapa fue más lenta?

Te lo deje comentado dentro del vs code.

https://github.com/kindred-98/Pipeline_Ventas/blob/main/docs/Incidencia_Pipeline.md

FASE 5 — Reflexión y comparativa

Pregunta	Tu respuesta
¿Por qué usamos <code>matplotlib.use('Agg')</code> en el pipeline y no en local?	Agg permite generar imágenes sin necesidad de una pantalla o entorno gráfico. En GitHub Actions no existe un servidor gráfico, así que cualquier intento de mostrar o renderizar una ventana fallaría.
¿Qué ventaja tiene <code>--cov-fail-under=80</code> en el pipeline respecto al equipo?	Obliga a que todo el código nuevo o modificado esté cubierto por tests antes de aceptarse en la rama principal. Es una forma automática de proteger el repositorio frente a regresiones o descuidos.
¿La IA generó correctamente los fixtures de pytest? ¿Tuviste que corregirlos?	La IA generó fixtures funcionales, pero fue necesario revisarlos para asegurar que cargaban el CSV real desde <code>data/ventas.csv</code> y que no inventaban datos. También se ajustaron algunos detalles para garantizar que los tests cubrieran rutas específicas, como errores por columnas faltantes o valores de <code>n</code> mayores que el número de productos.
¿Qué diferencia hay entre el reporte de cobertura en terminal y el HTML?	El reporte en terminal muestra un resumen rápido: porcentaje total, líneas cubiertas y líneas faltantes. El HTML, en cambio, permite navegar archivo por archivo y ver cada línea coloreada en verde o rojo.
Si añades una función nueva sin tests, ¿cómo afecta al pipeline?	El pipeline fallará automáticamente porque la cobertura bajará por debajo del umbral del 80%. Esto impide que la rama principal reciba código sin pruebas y obliga a escribir tests antes de fusionar los cambios. Es una protección automática que mantiene la calidad del proyecto a largo plazo.

Entrega: Sube todo a GitHub. El pipeline debe pasar con cobertura $\geq 80\%$ (badge verde). Pega el enlace del repo y captura del badge de cobertura en la tabla de arriba.